

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Principles of Programming Languages (Honors Module)

Applications of Answer Set Programming to Education

Version 0.0.1

HO CHI MINH CITY, 02/2026

Assignment prepared by Dr. Van-Giang Trinh (email, homepage). This document is for academic use only. Redistribution without permission is prohibited.

1 Introduction

Answer Set Programming (ASP) is a modern **declarative programming paradigm** used for knowledge representation and non-monotonic reasoning [2]. ASP is grounded in the stable model semantics proposed by Gelfond and Lifschitz (1988) [8]. In ASP, a problem is modeled as a logic program whose *stable models* correspond to valid solutions. With a declarative modeling language and efficient solvers [7], ASP is capable of encoding complex combinatorial problems (including NP-hard ones) in a natural and expressive way [2]. ASP has been successfully applied in diverse domains, from AI to industrial scheduling [2, 3, 4], affirming its role as a powerful modeling and reasoning framework for practical problems.

In education, ASP offers a range of promising applications [12, 1, 10, 11, 9]. It enables modeling of course knowledge as declarative rules, effectively forming a **knowledge base** of the subject. ASP can be used to **verify the correctness of exam problems and answers** by encoding constraints and checking whether proposed solutions satisfy them—producing counterexamples when they do not. Furthermore, ASP can help **automatically generate exercises and problem variants** that meet criteria such as difficulty or topic coverage [14]. Last, it supports the development of **personalized learning systems**, using student profiles and learning histories to recommend individualized learning paths [5, 13].

This mini-project focuses on educational applications of ASP to foster the following:

- **Problem modeling ability:** Students practice abstracting real-world problems (from various CS domains) into ASP programs with well-defined entities, relationships, and constraints.
- **Abstract and formal reasoning:** Students develop logical and mathematical thinking through rule-based modeling and correctness analysis.
- **Interdisciplinary applications of PPL:** Students integrate concepts from Principles of Programming Languages (PPL) with other CS courses, highlighting the role of formal logic and language semantics across domains.
- **Research and synthesis skills:** Students learn to read and synthesize related academic literature, comparing different approaches and designing solutions tailored to chosen problems.
- **Teamwork and academic writing:** Students collaborate in small teams (2–3 members), practice project planning, and produce well-structured reports in academic style.

2 Project requirements

- **Team structure:** Work in teams of 2–3 students
- **Topic scope:** Select **three different subjects** from the CSE curriculum, with **at least one being PPL**
- **ASP application per subject:** For each subject, design and implement one meaningful ASP application that leverages domain knowledge (not just **toy examples**). Each must include problem description, ASP modeling, and example stable models with explanation. Other possible subjects include:
 - Database systems
 - Data structures and algorithms
 - Digital systems
 - Operating systems
 - Parallel computing
 - Discrete structures for computing
 - Mathematical modeling
- **Cross-domain analysis:** Compare how ASP modeling differs across selected subjects. Highlight similarities, differences, and generalizable modeling patterns

3 Suggested ASP applications in PPL

Here are sample directions for ASP applications relevant to PPL:

1. **Type checking and inference** (see Chapter 8.8 of [6]): Use ASP to derive or verify expression types based on declarative type rules
2. **Scope and binding analysis** (see Chapter 4.3 of [6]): Encode variable bindings and free variable detection to catch scoping errors
3. **Operational semantics simulation** (see Chapter 2.5 of [6]): Represent small-step evaluation as transition rules and trace execution paths
4. **Evaluation strategy comparison** (see Chapter 6.1.3 of [6]): Model eager vs lazy evaluation and detect when order affects program behavior

4 Connection to exam evaluation research

One compelling research direction is using ASP for **exam and assessment verification**. In large-scale or online learning environments, automated grading and quality control of assessments are increasingly important.

ASP can be used to formally encode the logic of exam questions and reference answers. By running ASP solvers, we can find *counterexamples* that expose flaws in problem design or grading schemes. This enables automatic validation of correctness, consistency, and fairness of exam content.

In grading, ASP allows for logic-based comparisons between student answers and model answers. Instead of checking string outputs or predefined test cases, we can use ASP to verify whether student logic satisfies problem constraints. In symbolic execution or bounded model checking fashion, counterexamples to equivalence can serve as error explanations.

A general workflow may include: (1) encode the exam problem and evaluation criteria into ASP; (2) run a solver to find violations or generate counterexamples; (3) use findings to revise the question or diagnose student errors. ASP enables transparent, explainable, and reproducible assessment workflows—key goals of modern learning analytics systems.

5 Guidelines

5.1 Report

- **Structure:** Suggested structure includes: Introduction, Background (ASP and related subjects), Main content (three applications), Cross-domain analysis, Conclusion, References.
- **Format:** ≤ 15 pages (in total), PDF. Use figures and tables as needed, label clearly
- **Style:** Students are encouraged to present the report as a [scientific paper](#). The suggested LaTeX style is the EPTCS LaTeX style¹.
- **Language:** English or Vietnamese

5.2 Submission

Each group must prepare a ZIP file containing:

- Source code with README instructions
- A concise report

¹<https://style.eptcs.org/>

- All related data

This ZIP file must be named by the format “*PPL-HM-CSE252-{All member ID numbers separated by hyphens}.zip*” and submitted to the assignment section on the site of the course on BK-eLearning. Deadline for submission is **23h59, May 15, 2026 (HCMC time (GMT+7))**. Noting that for each group, **only the leader should make the submission**.

6 Evaluation and cheating judgement

6.1 Evaluation rubric

- **ASP modeling (30%)**: Correctness, completeness, and effectiveness of ASP programs. Models should run without error and solve nontrivial problems
- **Result analysis (30%)**: Interpretation of output, explanation of answer sets, discussion of interesting cases or limitations
- **PPL depth (15%)**: Significant consideration of main PPL concepts (semantics, typing, etc.) Use of correct terminology and theory
- **Cross-domain insights (10%)**: Comparative analysis across subjects. Highlight modeling challenges and insights from translating across domains
- **Presentation and writing (15%)**: Structure, clarity, formatting, citations. Writing must be academic and coherent
- **Bonus: Research innovation (up to 10%)**: Novel extensions, integration with other tools (e.g., SMT solvers), design of mini-frameworks or pipelines, etc.

6.2 Cheating judgement

Each group must develop its own results. While discussion of general ideas is allowed, direct sharing of code, ASP encodings, or experimental data between groups is strictly prohibited. A group will be considered as cheating if:

- There is an abnormal similarity among the reports. In this case, **ALL** submissions that are similar are considered as cheating. Therefore, the students of a group have to defend the work of their own group.
- They do not understand some parts written in their reports. Students can consult from any source, but make sure that they understand the meaning of everything they will have written. This rule also applies to source code.

If the work is found as cheating, students will be judged according to the university's regulations.

Positive advice: You are encouraged to discuss high-level modeling ideas with your peers and instructors. However, ensure that your encodings, analysis, data, and report are genuinely your own work. Understanding and explaining what you have built is more important than producing perfect results.

7 Changelogs

7.1 Version 0.0.1 (02/02/2026)

- First version

References

- [1] Mutsunori Banbara, Takehide Soh, Naoyuki Tamura, Katsumi Inoue, and Torsten Schaub. Answer set programming as a modeling language for course timetabling. *Theory Pract. Log. Program.*, 13(4-5):783–798, 2013.
- [2] Gerhard Brewka, Thomas Eiter, and Mirosław Truszczyński. Answer set programming at a glance. *Commun. ACM*, 54(12):92–103, 2011.
- [3] Esra Erdem, Michael Gelfond, and Nicola Leone. Applications of answer set programming. *AI Mag.*, 37(3):53–68, 2016.
- [4] Andreas A. Falkner, Gerhard Friedrich, Konstantin Schekotihin, Richard Taupe, and Erich Christian Teppan. Industrial applications of answer set programming. *Künstliche Intell.*, 32(2-3):165–176, 2018.
- [5] Davide Fuscà, Stefano Germano, Jessica Zangari, Marco Anastasio, Francesco Calimeri, and Simona Perri. A framework for easing the development of applications embedding answer set programming. In *PPDP*, pages 38–49. ACM, 2016.
- [6] Maurizio Gabbrielli, Simone Martini, and Saverio Giallorenzo. *Programming Languages: Principles and Paradigms, Second Edition*. Undergraduate Topics in Computer Science. Springer, 2023.
- [7] Martin Gebser, Benjamin Kaufmann, Roland Kaminski, Max Ostrowski, Torsten Schaub, and Marius Schneider. Potassco: The Potsdam answer set solving collection. *AI Commun.*, 24(2):107–124, 2011.
- [8] Michael Gelfond and Vladimir Lifschitz. The stable model semantics for logic programming. In *ICLP*, pages 1070–1080. MIT Press, 1988.

- [9] Susana Hahn, Torsten Schaub, Cedric Martens, Amadé Nemes, Henry Otunuya, Javier Romero, and Sebastian Schellhorn. Reasoning about study regulations in answer set programming. *Theory Pract. Log. Program.*, 24(4):790–804, 2024.
- [10] Daniela Inclezan. An application of answer set programming to the field of second language acquisition. *Theory Pract. Log. Program.*, 15(1):1–17, 2015.
- [11] Muhammed Kerem Kahraman and Esra Erdem. Personalized course schedule planning using answer set programming. In *PADL*, pages 37–45. Springer, 2019.
- [12] Johannes Oetsch, Jörg Pührer, Martin Schwengerer, and Hans Tompits. The system Kato: Detecting cases of plagiarism for answer-set programs. *Theory Pract. Log. Program.*, 10(4-6):759–775, 2010.
- [13] Henry Otunuya, Stefan Lindow, and Mariia Merzliakova. Reasoning over student study plans with answer set programming. In *55. Jahrestagung der Gesellschaft für Informatik, INFORMATIK 2025: The Wide Open - Offenheit von Source bis Science, Potsdam, Germany*, pages 1143–1153. Gesellschaft für Informatik, Bonn, 2025.
- [14] Petra Schwaiger and Burkhard Freitag. Using answer set programming for the automatic compilation of assessment tests. In *ICLP*, pages 300–314. Springer, 2006.